Informe — Laboratorio 4

Excepciones, archivos, persistencia (JSON/CSV) y POO intermedia en Python

Aprendiz	Anthony Córdova Calderón
Ficha	3388087
Laboratorio	Laboratorio 4 — Gestor de inventario persistente
Usuario de GitHub	skyyleerr
Repositorio	https://github.com/skyyleerr/python-laboratorios
Carpeta	laboratorio-04/ (proyecto en laboratorio-04/gestor_inventario/)
Informe en el repositorio	laboratorio-04/informe/Informe_Laboratorio_04.pdf

Es el mismo repositorio de los laboratorios 01, 02 y 03, que se conservan sin cambios en sus carpetas. La versión del Laboratorio 3 también se guardó dentro de este laboratorio en `gestor_inventario/version_lab3/` para comparar ambas implementaciones.

1. Nuevas funcionalidades incorporadas

El gestor del Laboratorio 3 pasó de perder todos los datos al cerrarse a **conservar el inventario entre ejecuciones**. Se mantuvieron las operaciones anteriores (registrar, listar, buscar, actualizar, eliminar, sin stock y resumen) y se agregó:

- **Persistencia en JSON** (`almacenamiento.py`): el inventario se carga al iniciar y se guarda tras cada operación aceptada y desde la opción 8.
- **Serialización de objetos:** `Producto.a_diccionario()` y `Producto.desde_diccionario()`; `Inventario.exportar_datos()` y `Inventario.desde_datos()`.
- **Protección ante archivos dañados:** JSON con sintaxis incorrecta, estructura inesperada, registros inválidos o códigos repetidos no se cargan ni se sobrescriben; la aplicación pasa a *modo solo consulta*.
- **Exportación CSV** (`reportes.py`, opción 9) con la columna `valor_inventario = precio × cantidad`.
- **Manejo de errores de escritura:** si falla el guardado se avisa que el cambio queda solo en memoria y no se anuncia «guardado».
- **Rutas robustas** con `Path(__file__)`: el programa encuentra los mismos archivos aunque se ejecute desde otra carpeta.
- Menú ampliado con las opciones 8 (guardar) y 9 (exportar CSV).

Estructura de módulos y responsabilidades:

Módulo	Responsabilidad	¿Conoce archivos?
producto.py (64 líneas)	Reglas del producto y conversión a/desde diccionario.	No
inventario.py (59)	Colección de productos; exporta datos y se reconstruye desde datos validados.	No
almacenamiento.py (47)	Guardar y cargar el JSON; define ErrorAlmacenamiento.	Sí: datos/productos.json
reportes.py (26)	Exportar el CSV; no modifica existencias.	Sí: reportes/inventario.csv

Módulo	Responsabilidad	¿Conoce archivos?
main.py (151)	Menú, lectura de datos, coordinación y mensajes. Único módulo con input().	Indirectamente

2. Almacenamiento JSON, exportación CSV y manejo de excepciones

2.1 Almacenamiento JSON

Guardar: `guardar_inventario(inventario, ruta)` obtiene los datos con `inventario.exportar_datos()` (lista de diccionarios), crea la carpeta `datos/` si hace falta y escribe con `json.dump(..., ensure_ascii=False, indent=4, allow_nan=False)` en UTF-8. La escritura es **atómica**: primero se escribe un archivo temporal en la misma carpeta, se fuerza a disco (`os.fsync`) y solo entonces `replace()` sustituye al original. Si algo falla antes, el JSON anterior queda intacto y el temporal se elimina.

Cargar: `cargar_inventario(ruta)` distingue tres situaciones:

Situación	Excepción / camino	Resultado
El archivo no existe	FileNotFoundError	Inventario vacío; se creará al guardar
JSON con sintaxis incorrecta o codificación inválida	json.JSONDecodeError / UnicodeDecodeError → ErrorAlmacenamiento	Se informa; modo solo consulta; archivo intacto
JSON válido pero con datos inválidos (no es lista, faltan campos, categoría o precio incorrectos, código repetido)	ValueError / TypeError → ErrorAlmacenamiento	Se rechaza la carga completa; archivo intacto
JSON válido y correcto	—	Se reconstruyen todos los Producto

Reconstrucción sin carga parcial: `Inventario.desde_datos()` crea un *inventario temporal*, construye cada `Producto` con el constructor (así se aplican todas las validaciones) y rechaza duplicados. Solo si todos los registros son correctos devuelve el inventario; si uno falla se lanza la excepción y no se usa nada de lo leído.

```
temporal = cls()
for registro in datos:
    producto = Producto.desde_diccionario(registro)          # valida con el constructor
    if not temporal.registrar_producto(producto):
        raise ValueError(f"Código duplicado en archivo: {producto.codigo}")
return temporal
```

2.2 Exportación CSV

`exportar_csv(inventario, ruta)` obtiene los productos por la interfaz pública `listar_productos()`, crea `reportes/` y escribe con `csv.DictWriter` (`newline=""`, UTF-8) las columnas `codigo, nombre, categoria, precio, cantidad, valor_inventario`. El valor de cada fila viene de `producto.calcular_valor()` (precio × cantidad), de modo que la regla vive en una sola clase. El CSV es una **exportación**: el JSON sigue siendo la única fuente de datos persistente.

2.3 Manejo de excepciones

Dónde	Excepción	Tratamiento
main.py (leer_precio, leer_entero)	ValueError al convertir	Mensaje y nueva oportunidad; el ciclo no se interrumpe
producto.py	ValueError / TypeError (raise)	El objeto no se crea con datos inválidos; main.py muestra «Datos rechazados»
almacenamiento.py (guardar)	OSError	Se elimina el temporal y se relanza; main.py avisa que el cambio queda solo en memoria
almacenamiento.py (cargar)	JSONDecodeError, ValueError, TypeError	Se convierten en ErrorAlmacenamiento; main.py activa el modo solo consulta
main.py (ciclo principal)	ValueError, TypeError, OSError	Se informan sin cerrar la aplicación

No se usa `except Exception` para «arreglar» errores: se capturan solo las excepciones esperadas y se evita el patrón que sustituiría un archivo dañado por una lista vacía.

3. Estado del proyecto, dificultades, soluciones y pendientes

Estado: completo respecto a lo solicitado. Las diez opciones del menú funcionan; las prácticas 4.1 a 4.4 están desarrolladas; se resolvió el reto de depuración (siete casos) y se incluyó una mejora de ampliación (guardado atómico mediante archivo temporal).

Dificultad	Solución
Evitar una carga parcial si un registro del JSON es inválido.	Construir primero un inventario temporal en Inventario.desde_datos() y usarlo solo si todo es válido.
Distinguir archivo inexistente de archivo dañado.	FileNotFoundError devuelve un inventario vacío; JSONDecodeError y datos inválidos se convierten en ErrorAlmacenamiento.
Impedir que la aplicación sobrescriba un JSON dañado al guardar (por ejemplo, al salir).	Modo solo consulta: registrar, actualizar, eliminar, guardar y el guardado final de la opción 0 quedan bloqueados.
Probar la aplicación sin tocar datos reales.	pruebas_obligatorias.py simula input() y usa un directorio temporal. Como la ruta por defecto se fija al definir la función, las pruebas pasan la ruta explícita con functools.partial.
Formato de salida distinto del enunciado (menú con «10. Salir (también 0)», importes con separadores de miles).	Corregido en el commit 101d23e: menú con «0. Salir» e importes como $1740000.0.
Precios escritos con punto de miles (85.000), que Python leería como 85.0.	convertir_precio() de main.py interpreta 85.000 como 85000 y 85.000,50 como 85000.5 (heredado del Lab 3 y probado).

Funcionalidades pendientes o limitaciones conocidas:

- No hay copia de seguridad del JSON antes de sobrescribirlo (solo escritura atómica).
- En modo solo consulta no hay reparación automática: hay que corregir o restaurar el archivo a mano y reiniciar. Es lo que pide el enunciado.
- No hay reporte por categoría ni exportación de agotados a un CSV independiente (ampliaciones opcionales).
- Cada exportación CSV reemplaza la anterior; el reporte no se actualiza solo.
- El código del producto se normaliza a mayúsculas, así que «p001» y «P001» son el mismo producto.
- Se considera un único usuario: no hay bloqueo si dos procesos escriben el mismo archivo a la vez.

4. Seguimiento de commits y decisiones técnicas

Hashes reales del repositorio, en orden cronológico. La tabla relaciona cada commit con una decisión, su motivo y la comprobación asociada.

Commit	Cambio realizado	Razón del cambio	Comprobación
9718822	chore: .gitignore para pycache y datos generados	Evitar versionar caché y datos generados (datos/, reportes/).	git status limpio tras ejecutar
8e37ff1	feat(lab04): practica 4.1 excepciones, raise, else y finally	Base para tratar errores: try/except/else/finally y raise.	Cada ejercicio se ejecutó con entradas válidas e inválidas
b910053	feat(lab04): practica 4.2 archivos de texto, modos y pathlib	Aprender archivos de texto y rutas con pathlib antes de usar JSON.	Archivos de ejemplo generados y conservados
49620af	feat(lab04): practica 4.3 JSON, objetos a diccionarios, JSON danado y CSV	JSON, CSV y la diferencia entre objeto y diccionario.	lectura_protegida.py respeta un JSON dañado
c890c96	feat(lab04): practica 4.4 herencia, polimorfismo y composicion	Herencia, polimorfismo y composición en ejemplos aparte.	Ejemplos ejecutados; el gestor usa composición
736cbfa	chore(gestor): conservar copia del gestor del laboratorio 3 en version_lab3	Conservar el Lab 3 para comparar versiones sin sobrescribirla.	Archivos idénticos a la entrega del Lab 3
f5198a9	feat(gestor): Producto con a_diccionario() y desde_diccionario() validando en el constructor	Convertir objetos a datos serializables y reconstruirlos validando.	Ida y vuelta de un producto da el mismo diccionario (test_gestor)
a412800	feat(gestor): Inventario con exportar_datos() y desde_datos() sin carga parcial	Exponer datos sin la lista interna y cargar sin carga parcial.	Prueba de código repetido y registro inválido: la carga se rechaza completa
4969b24	feat(gestor): almacenamiento.py con guardado atomico y carga que no pisa archivos danados	Separar la persistencia; escritura atómica; no pisar archivos dañados.	JSON mal formado queda idéntico tras el intento de carga
e1595fb	feat(gestor): reportes.py exporta CSV con valor_inventario = precio * cantidad	Módulo propio para el CSV; valor = precio × cantidad vía calcular_valor().	Filas 340000.0 / 0.0 / 1400000.0
43f1006	feat(gestor): main.py guarda tras cada operacion aceptada y entra en solo consulta si el JSON es invalido	Guardar tras cada operación aceptada y bloquear escritura si el JSON es inválido.	Sesiones de reinicio en pruebas_obligatorias.py
f12f799	test(gestor): pruebas unitarias de serializacion, persistencia, JSON danado y CSV	Proteger reglas de Producto, Inventario, JSON y CSV.	10 pruebas OK
101d23e	fix(gestor): menu con opcion 0 y formatos de salida segun el enunciado	Alinear el menú y los formatos con el enunciado (corrección real: 9 inserciones, 8 eliminaciones en main.py).	Pruebas numéricas muestran $1740000.0 y $1965000.0
125204b	test(gestor): pruebas de sesion completa con reinicio, archivos danados y error de escritura	Probar la tabla de pruebas obligatorias, incluido el reinicio y el error de escritura.	16 pruebas OK

Commit	Cambio realizado	Razón del cambio	Comprobación
71a0d60	docs(gestor): reto de depuracion con siete problemas reproducidos	Reproducir los siete problemas del reto y documentar su causa.	gestor_depuracion.py ejecuta los siete casos
4bffb56	docs(lab04): READMEs del proyecto y del laboratorio	Documentar ejecución, responsabilidades y pruebas.	Comandos del README ejecutados
463bf99	docs: README principal con los cuatro laboratorios y su evolucion	Presentar los cuatro laboratorios y su evolución.	Enlaces a las cuatro carpetas

Transparencia sobre el historial. El historial se armó a partir de los archivos del laboratorio ya desarrollados, con ayuda de Claude (los commits incluyen la línea Co-Authored-By). Por eso las fechas son las de su creación. El commit `101d23e` sí corresponde a una corrección real: el commit anterior (`43f1006`) contiene la versión de `main.py` previa al ajuste.

5. Evidencias de pruebas

Comando	Resultado
python -m unittest -v	10 pruebas, OK
python pruebas_obligatorias.py	16 pruebas, OK
python gestor_depuracion.py	Siete problemas reproducidos y corregidos
python main.py (dos ejecuciones reales, la segunda desde otra carpeta)	Se recuperan los datos (ver 5.1)

5.1 Persistencia y recuperación (dos procesos distintos)

Primera ejecución sin `datos/productos.json`: se registran P001, P002 y P003. Segunda ejecución, iniciada desde la carpeta raíz (`cd /`) para comprobar la ruta:

```
1ª ejecución:  Productos recuperados: 0
               Producto registrado.  Guardado en: .../datos/productos.json   (x3)
               Programa finalizado.
2ª ejecución:  Productos recuperados: 3
   Opción 7 -> Productos registrados: 3
               Unidades disponibles: 6
               Valor del inventario: $1740000.0
               Categorías utilizadas: 2 ['Componentes', 'Periféricos']
   Opción 6 -> P002 - Mouse - Periféricos - $45000.0 - 0 unidades - valor $0.0
   Opción 4 -> P002 = 5          Opción 9 -> Reporte exportado: .../reportes/inventario.csv
```

Resultados esperados por el laboratorio: 3 productos, 6 unidades y $1740000.0; tras cambiar el Mouse a 5, 11 unidades y $1965000.0 (la prueba automática `test_18_resumen_y_prueba_numerica_con_reinicio` verifica ambos con reinicio entre sesiones).

5.2 Reporte CSV generado

```
codigo,nombre,categoria,precio,cantidad,valor_inventario
P001,Teclado,Periféricos,85000.0,4,340000.0
P002,Mouse,Periféricos,45000.0,5,225000.0
P003,Monitor,Componentes,700000.0,2,1400000.0
```

La fila de P002 muestra 225000.0, como pide el enunciado; la suma de la columna es 1965000.0, igual al resumen.

5.3 Prueba de archivo dañado (sobre una copia)

Se guardó un inventario válido, se hizo una copia del JSON y se eliminó una coma del archivo usado por la aplicación:

```
No fue posible cargar el inventario: JSON dañado en .../datos/productos.json:
    Expecting ',' delimiter: line 9 column 5 (char 158)
AVISO: archivo persistente inválido. Modo de solo consulta; no se guardará sobre el archivo.
Opción 2 -> No hay productos registrados.
Opción 1 -> Operación bloqueada hasta resolver el archivo JSON.
Opción 8 -> Guardado bloqueado para proteger el archivo inválido.
Opción 0 -> Programa finalizado.
cmp datos/productos.json danado.json  ->  idénticos (no fue reemplazado por [])
Se restauró la copia válida y se reinició:  Productos recuperados: 3
```

5.4 Tabla de pruebas obligatorias

Situación	Resultado	Prueba
Primera ejecución, JSON inexistente	Inventario vacío sin error	test_01
Guardar dos productos / cerrar y reiniciar	JSON con ambos registros; se recuperan	test_02_03
Código duplicado	Rechazado; el JSON no se duplica	test_04
Categoría desconocida	Rechazada	test_05
Precio negativo, nan o inf	Rechazados (3 mensajes) y se pide de nuevo	test_06_07
Cantidad negativa / cero	Rechazada / aceptada como sin stock	test_08_09
Actualizar y eliminar	Persisten tras reiniciar	test_10, test_11
Texto no numérico	Mensaje y nueva oportunidad	test_12
JSON con sintaxis incorrecta	Se informa; archivo intacto; operaciones bloqueadas	test_13
JSON con estructura inesperada	Carga rechazada; archivo intacto	test_14
JSON con código repetido	Sin carga parcial; archivo intacto	test_15
Error de escritura	«Queda solo en memoria»; sin «Guardado en:»	test_16
Exportar CSV	Encabezados y valores 340000 / 225000 / 1400000	test_17
Resumen y prueba numérica	$1740000.0 y $1965000.0 tras reiniciar	test_18
Seleccionar 0	Salida controlada	test_19

6. Evolución del proyecto: Laboratorio 01 → Laboratorio 04

Laboratorio	Qué se construyó	Cómo se representaban los datos	Qué ocurría al cerrar	Principal avance
01 — Fundamentos	Cotizador y programas lineales (perfil, promedio, inventario simple).	Variables sueltas	Se perdía todo	Entrada/salida y cálculos
02 — Decisiones y colecciones	Gestor de inventario procedural en un archivo con menú.	Lista de diccionarios + tupla de categorías	Se perdía todo	Condiciones, ciclos, colecciones, comprensiones
03 — Funciones y POO	Gestor con Producto, Inventario y main.py (31 pruebas).	Objetos Producto en un Inventario	Se perdía todo	Encapsulamiento, validación en el constructor, separación de responsabilidades
04 — Persistencia	Gestor con almacenamiento.py y reportes.py (26 pruebas).	Objetos + JSON en disco + CSV de reportes	Se conserva en datos/productos.json	Excepciones, archivos, serialización, protección de datos dañados

Del Laboratorio 3 al 4, el código de dominio casi no cambió: `producto.py` pasó de 81 a 64 líneas e `inventario.py` de 74 a 59 (se compactaron), y se les añadió solo la conversión a datos. Las nuevas responsabilidades (persistencia y reportes) se pusieron en módulos distintos: así `Producto` e `Inventario` siguen sin conocer archivos y pueden probarse sin disco. El gestor usa **composición** (el inventario contiene productos; main.py usa funciones de almacenamiento y reportes); no se introdujo herencia porque no hay una relación de especialización.

7. Respuestas a las preguntas de análisis situacional

Pregunta 1

¿Qué harías si el programa se cierra inesperadamente y, al volver a ejecutarlo, los productos registrados ya no aparecen?

Qué haría. Primero comprobaría si los cambios llegaron a guardarse, luego revisaría la ruta del archivo y por último el proceso de carga.

Por qué. Hay tres causas posibles: nunca se guardó (los cambios quedaron en memoria), se guardó en otra ubicación porque la ruta era relativa, o el archivo existe pero la carga falló o se ignoró.

Cómo lo implementaría. (1) Buscar `datos/productos.json` en la carpeta del proyecto y abrirlo: si falta o está vacío, el problema es el guardado; revisar que `main.py` llame a `guardar_inventario` tras cada operación aceptada y que el mensaje «Guardado en:» muestre la ruta. (2) Imprimir `RUTA_JSON`: en el proyecto se construye con `Path(__file__).resolve().parent`, por lo que no depende de la carpeta de ejecución. (3) Si el archivo tiene datos, revisar que `cargar_inventario()` no falle y que el programa no entre en modo solo consulta. En el proyecto, un fallo de escritura se informa como «queda solo en memoria» y nunca como guardado.

```
RUTA_JSON = BASE_DIR / "datos" / "productos.json"      # almacenamiento.py
...
inventario = cargar_inventario()                       # main.py, al iniciar
print("Productos recuperados:", len(inventario.listar_productos()))
```

Cómo lo comprobaría. Registrar tres productos, cerrar, y ejecutar de nuevo `python main.py` desde otra carpeta (`cd ..`): aparece «Productos recuperados: 3» (evidencia 5.1). La prueba

`test_02_03_guardar_y_reiniciar` repite el ciclo con dos sesiones y `test_guardar_recuperar_desde_directorio_distinto` verifica una ruta distinta.

Pregunta 2

¿Qué harías si productos.json existe, pero tiene una coma faltante y el programa no puede leerlo?

Qué haría. Leería el mensaje de la excepción para ubicar el error, no reemplazaría el archivo y lo repararía o restauraría a mano antes de volver a abrir la aplicación.

Por qué. Un JSON dañado todavía contiene datos recuperables. Sustituirlo automáticamente por [] los destruiría. Un archivo inexistente (primera ejecución) es otro caso distinto, en el que sí se parte de un inventario vacío.

Cómo lo implementaría. Capturar `json.JSONDecodeError` (el mensaje indica línea y columna: «Expecting ',' delimiter: line 9 column 5»), convertirlo en `ErrorAlmacenamiento` y no guardar nada. En `main.py` la aplicación entra en modo solo consulta: registrar, actualizar, eliminar, guardar y el guardado final de la opción 0 quedan bloqueados. Procedimiento: hacer una copia del archivo, abrirlo en VS Code (que marca el error de sintaxis), agregar la coma o restaurar una copia válida y reiniciar.

```
except (json.JSONDecodeError, UnicodeDecodeError) as error:
    raise ErrorAlmacenamiento(f"JSON dañado en {ruta}: {error}") from error
```

Cómo lo comprobaría. Se eliminó una coma de una copia: el programa informó el error y bloqueó la escritura; `cmp` mostró el archivo idéntico (evidencia 5.3). Al restaurar la copia válida se recuperaron los 3 productos. Automatizado en `test_13_json_con_sintaxis_incorrecta` y `test_json_mal_formado_no_se_pisa`.

Pregunta 3

¿Qué harías si el inventario guarda correctamente un producto, pero al recuperar el JSON aparecen dos registros con el mismo código?

Qué haría. Comprobaría la unicidad en el momento de la carga (y no solo al registrar) y haría que cualquier duplicado rechace la carga completa.

Por qué. El archivo puede editarse a mano o provenir de otra versión, así que no se puede confiar en que cumpla las reglas. Aplicar las validaciones solo al registrar deja una puerta abierta.

Cómo lo implementaría. La unicidad es responsabilidad de `Inventario`, que ya impide duplicados en `registrar_producto()`. `Inventario.desde_datos()` construye un inventario temporal y registra cada producto allí; si uno es duplicado o inválido lanza `ValueError` y `cargar_inventario()` lo convierte en `ErrorAlmacenamiento`. Los objetos se reconstruyen con `Producto.desde_diccionario()` (constructor con validaciones), no con asignaciones directas, por lo que también se detectan precios negativos o categorías desconocidas.

```
temporal = cls()
for registro in datos:
    producto = Producto.desde_diccionario(registro)
    if not temporal.registrar_producto(producto):
        raise ValueError(f"Código duplicado en archivo: {producto.codigo}")
return temporal      # solo si todos los registros fueron válidos
```

Cómo lo comprobaría. Prueba `test_registro_invalido_o_duplicado_rechaza_carga_completa` y `test_15_codigo_repetido_en_json_sin_carga_parcial`: se escribe un JSON con dos «P1», la carga falla, no aparece ningún producto y el archivo queda sin cambios. En `gestor_depuracion.py` (caso 4) se ve el error de cargar sin validar: 2 productos con el mismo código.

Pregunta 4

¿Qué harías si un reporte CSV muestra el precio unitario en la columna del valor total de inventario, aunque cada producto tenga varias unidades?

Qué haría. Revisaría el cálculo de la columna `valor_inventario` y confirmaría que usa precio × cantidad.

Por qué. Usar solo el precio ignora las existencias y subestima el valor. La regla del valor de un producto pertenece a `Producto.calcular_valor()`, no a cada informe.

Cómo lo implementaría. En `reportes.py` la fila se arma con `producto.calcular_valor()`, que devuelve `self._precio * self._cantidad`. El reporte solo lee datos por la interfaz pública del inventario y no modifica existencias.

```
filas.append({"codigo": producto.codigo, ..., "precio": producto.precio,
              "cantidad": producto.cantidad,
              "valor_inventario": producto.calcular_valor()})
```

Cómo lo comprobaría. Con los datos conocidos: P001 = 85000 × 4 = 340000, P002 = 45000 × 5 = 225000, P003 = 700000 × 2 = 1400000; la suma, 1965000, coincide con el resumen (evidencia 5.2). Pruebas `test_csv_encabezados_y_valor` y `test_17_exportar_csv`. En `gestor_depuracion.py` (caso 6), sumar precios da 785000.0 y el cálculo correcto 1740000.0.

Pregunta 5

¿Qué harías si, después de separar el programa en varias clases y módulos, una corrección al guardado provoca que el inventario deje de cargar datos?

Qué haría. Usaría el historial de Git para encontrar el commit que introdujo el fallo, reproducirlo con una prueba, corregirlo y documentarlo.

Por qué. En un repositorio con commits pequeños, cada cambio al guardado queda aislado; se puede comparar la versión que funcionaba con la que falla y entender la causa en vez de adivinar.

Cómo lo implementaría. (1) `git log --oneline -- almacenamiento.py` para ver qué commits tocaron el módulo. (2) `git diff -- almacenamiento.py` (o la comparación de commits en GitHub) para ver el cambio. (3) `git bisect` para localizar el commit si hay varios candidatos. (4) Escribir una prueba que guarde y vuelva a cargar, y que falle con el cambio defectuoso. (5) Corregir con un commit `fix:` cuyo mensaje explique la causa. Si hace falta, se puede deshacer con `git revert` sin perder historial. En este proyecto, `almacenamiento.py` tiene su propio commit (`4969b24`) y las pruebas de ida y vuelta (`f12f799`) detectarían cualquier cambio que rompa la carga.

```
git log --oneline -- laboratorio-04/gestor_inventario/almacenamiento.py
git diff 4969b24 HEAD -- laboratorio-04/gestor_inventario/almacenamiento.py
python -m unittest -v        # test_guardar_recuperar_desde_directorio_distinto
```

Cómo lo comprobaría. Tras la corrección ejecutaría las 26 pruebas y la sesión de reinicio manual (guardar, cerrar, abrir). La evidencia queda en el mensaje del commit, en las pruebas añadidas y en este informe. Como ejemplo real de corrección documentada, el commit `101d23e` muestra el antes y el después de `main.py` con un diff de 9 inserciones y 8 eliminaciones.

8. Conclusión

El gestor de inventario ahora conserva su información entre ejecuciones, protege los datos existentes ante archivos dañados o escrituras fallidas y genera reportes CSV correctos. Cada responsabilidad nueva tiene su propio módulo, el dominio no conoce el sistema de archivos y las 26 pruebas automáticas, junto con las pruebas manuales de reinicio y de JSON dañado, comprueban el comportamiento. Repositorio: https://github.com/skyyleerr/python-laboratorios.